

Parallel Image Filter and Processor Using Hybrid MPI and OpenMP: A Pthreads, OpenMP, and MPI Comparative Study

Sameer Babar

2023-CS-162

Department of Computer Science

University of Engineering and Technology, Lahore

Parallel and Distributed Computing Lab, Spring 2026

Instructor: Mr. Waqas Ali

Abstract—High-resolution image filtering, including Gaussian blur, Sobel edge detection, unsharp-mask sharpening, and RGB-to-grayscale conversion, is an embarrassingly parallel, compute-bound workload that scales poorly on a single core. This paper presents a comparative implementation of four versions of the same filtering pipeline: a sequential baseline, a Pthreads version, an OpenMP version, and a distributed-memory MPI version using row-wise decomposition with explicit halo exchange. All four produce byte-identical output, 96 of 96 configurations checked, on two real datasets: photographs resized to nine resolutions from 4,096 to 18,662,400 pixels, and a balanced 5,000-photograph subset of Imagenette. On a 4-core, 8-thread Intel Core i5-8365U machine, OpenMP achieves the best result, a $3.66\times$ speedup for Sobel edge detection at 4 threads. All three parallel models peak near 4 workers and regress at 8, since the machine has only 4 physical cores behind its 8 logical ones; this regression turns severe when paid thousands of times, as in the 5,000-photograph experiment, where MPI falls to 25 times slower than sequential at 8 ranks because each small image still pays a full communication round trip. Pthreads never clearly beats sequential at any scale tested. We compare programming effort via source line count and discuss the causes behind these limits: thread-pool reuse, MPI’s per-call communication overhead, and core oversubscription.

Index Terms—parallel computing, MPI, OpenMP, Pthreads, image processing, convolution, speedup, strong scaling, weak scaling

I. Introduction

Image filtering operations – smoothing, edge detection, sharpening, and color-space conversion – underpin a large fraction of modern computer vision and image-processing pipelines [1]. Each of these operations is, in the language of Flynn’s taxonomy [2], naturally suited to SIMD/MIMD parallel execution: the output value of one pixel depends only on a small, fixed-size neighborhood of input pixels, so disjoint regions of an image can be filtered completely independently of one another. At the same time, the work involved grows directly with image resolution, which has increased far faster than single-core clock speed over the last two decades. A single 5×5 Gaussian-blur pass over one 4K (3840×2160) grayscale frame requires $3840\times 2160\times 5\times 5 = 207,360,000$ multiply-accumulate operations; a real-time pipeline that must apply several such filters to

a stream of frames – as in medical-imaging enhancement, satellite remote sensing, social-media photo filters, or the perception stack of an autonomous vehicle – cannot rely on a single CPU core to keep up [5].

Parallel and distributed computing offers a direct answer: because neighboring-pixel dependencies are confined to a small kernel radius, an image can be partitioned into independent row-strips (or blocks) that are processed concurrently, with only a thin border of “halo” pixels needing to cross a partition boundary [6], following the general data-decomposition principles laid out in standard parallel computing texts [7], [8]. This paper implements and benchmarks that idea across three distinct parallel programming models – Pthreads, OpenMP, and MPI – applied to the same four filters, and compares them against a common sequential baseline. Our specific contributions are:

- A from-scratch C implementation of Gaussian blur, Sobel edge detection, unsharp-mask sharpening, and BT.601 grayscale conversion in four versions (sequential, Pthreads, OpenMP, MPI) that are verified to produce byte-identical output.
- A benchmark dataset built from two real, license-clean photographs rather than synthetic random pixels – a methodological point we argue matters for convolution filters, whose output is meaningless on spatially uncorrelated noise – resized to nine resolutions spanning 10^3 to nearly 2×10^7 pixels using a custom bilinear resizer.
- Strong-scaling, weak-scaling, and problem-size-sweep measurements on real hardware, plus a quantitative comparison of programming effort across the three parallel models.
- A second, complementary real-world experiment on a balanced 5,000-photograph subset of Imagenette [29] (an ImageNet-derived dataset, not a resized version of the same two source photographs), which exposes a core-oversubscription bottleneck far more dramatically than any single-image experiment: at 8

workers, the same per-call overhead that is only a minor effect on one large image is paid out 5,000 times.

The remainder of this paper is organized as follows. Section II describes the sequential baseline and its complexity. Section III details the three parallel designs. Section IV describes the experimental setup. Section V reports strong-scaling, weak-scaling, and timing results. Section VI discusses bottlenecks and programming effort, and Section VII concludes.

II. Sequential Baseline

All four filters share the same structure: for every output pixel, read a small, fixed neighborhood of input pixels, combine them with a constant kernel, and clamp the result to $[0, 255]$. Algorithm 1 gives the generic convolution loop shared by Gaussian blur ($K = 5$, kernel sum 273), Sobel edge detection ($K = 3$, two kernels combined via gradient magnitude), and unsharp-mask sharpening ($K = 3$). Grayscale conversion has no neighborhood dependency: each output pixel is an independent weighted sum of the three input channels at the same coordinate ($Y = 0.299R + 0.587G + 0.114B$), so it is the special case $K = 1$.

Algorithm 1 Generic $K \times K$ convolution filter

```

1: procedure Convolve(in, out, W, H, kernel, K)
2:   half  $\leftarrow \lfloor K/2 \rfloor$ 
3:   for y  $\leftarrow$  half to H - half - 1 do
4:     for x  $\leftarrow$  half to W - half - 1 do
5:       sum  $\leftarrow$  0
6:       for ky  $\leftarrow$  0 to K - 1 do
7:         for kx  $\leftarrow$  0 to K - 1 do
8:           sum  $\leftarrow$  sum + in[y + ky - half][x +
             kx - half] · kernel[ky][kx]
9:         end for
10:      end for
11:      out[y][x]  $\leftarrow$  Clamp(sum/norm, 0, 255)
12:    end for
13:  end for
14:  CopyOrZeroBorder(in, out, W, H, half)
15: end procedure

```

Time complexity. The double loop visits every interior pixel exactly once and performs $O(K^2)$ work per pixel, giving $O(W \cdot H \cdot K^2)$ for the convolution filters and $O(W \cdot H)$ for grayscale conversion. Since $K \in \{3, 5\}$ is a small compile-time constant in our implementation, this is $O(N)$ in the number of pixels $N = W \cdot H$ for all four filters in practice.

Space complexity. Each filter allocates one input and one output buffer of size $O(W \cdot H)$ (or $O(3 \cdot W \cdot H)$ for the RGB input to grayscale conversion); the kernel weights are $O(K^2) = O(1)$. A separate output buffer is required because computing an interior output pixel reads neighboring input pixels that would otherwise be

overwritten, so the filters cannot be done in place. Total space is $O(W \cdot H)$.

Measured baseline. On our test machine (Section IV), the sequential baseline on a real 3840×2160 photograph measures $T_{\text{serial}} = 0.0703$ s for Gaussian blur, 0.0469 s for Sobel, 0.0116 s for sharpening, and 0.0258 s for grayscale conversion – each the minimum of three repeated trials, timed with `clock_gettime(CLOCK_MONOTONIC)` rather than `clock()`, since the latter measures CPU time rather than wall-clock time and would silently misreport once the same measurement code is reused in the multi-threaded versions.

III. Parallel Design

All three parallel versions use the same underlying decomposition strategy – horizontal row-strips – but differ fundamentally in how a worker obtains the neighbor data it needs, and how completion is synchronized, following the general shared-memory/distributed-memory design patterns described in standard parallel-programming references [9], [10].

A. Pthreads

Implemented directly against the POSIX threads standard [22], with no third-party threading library.

Decomposition. Because Pthreads workers share one address space, decomposition is purely a scheduling decision: the H output rows are split into P contiguous, near-equal blocks (the first $H \bmod P$ blocks get one extra row), and one worker thread is created per block.

Synchronization. Since every thread can read any row of the shared, read-only input buffer directly, no halo copying is needed – a thread responsible for output rows $[r_0, r_1)$ simply reads input rows $[r_0 - \text{half}, r_1 + \text{half})$ in place. Because each thread writes only to its own disjoint slice of the output buffer, no mutexes, atomics, or condition variables are required anywhere in the filter loop; the only synchronization point is the implicit barrier formed by `pthread_join()` on every worker before the border pixels are patched up and the result is used.

```

typedef struct { const unsigned char *in; unsigned char *out;
                 int width, height, row_start, row_end; } task_t;

for (i = 0; i < nthreads; i++) {
  tasks[i] = (task_t){in, out, width, height, row, row + rows};
  pthread_create(&tid[i], NULL, worker_gaussian, &tasks[i]);
  row += rows;
}
for (i = 0; i < nthreads; i++) pthread_join(tid[i], NULL);

```

A new pool of OS threads is created and joined for every filter call, since our implementation favors a direct, explicit mapping from the sequential algorithm rather than introducing a persistent thread pool; Section VI quantifies the cost of that choice.

B. OpenMP

Implemented against the OpenMP API specification [19], [20], following the directive-based style described in [21].

Decomposition. Identical row-strip decomposition to Pthreads, but expressed declaratively: the compiler and OpenMP runtime divide the outer y loop into contiguous chunks across `num_threads()` threads via `schedule(static)`, rather than the programmer computing row ranges by hand.

Synchronization. As with Pthreads, every iteration writes a disjoint output pixel, so no locks are needed; the only synchronization is the implicit barrier at the end of the parallel region. Critically, the GCC OpenMP runtime (libgomp) maintains a thread pool that can be reused across the four separate parallel regions (one per filter) within a single process, which our results show matters considerably in practice.

```
#pragma omp parallel for schedule(static) num_threads(nthreads)
for (int y = half; y < height - half; y++) {
    for (int x = half; x < width - half; x++) {
        /* identical convolution body to Algorithm 1 */
    }
}
```

C. MPI

Decomposition. Rank 0 reads the input image from disk and uses `MPI_Scatterv` to distribute row-strips to every rank, with strip sizes computed the same way as the thread-block split above. Unlike the shared-memory models, MPI ranks do not share an address space, so a rank cannot simply read a neighboring rank’s rows.

Synchronization / communication. Each convolution filter therefore needs an explicit halo exchange: before filtering, every rank sends its first half core rows to rank $i - 1$ and its last half core rows to rank $i + 1$, and receives the corresponding rows back, via paired `MPI_Sendrecv` calls (Listing 1). Ranks at the true top/bottom edge of the image have no neighbor on that side; rather than leave the halo undefined, we mirror the nearest real row into it so the same interior convolution loop can run unmodified for every rank – but we then overwrite those true image-edge output rows with a direct copy of the input (or zero, for Sobel) once results are gathered back to rank 0, exactly matching the border policy of the other three implementations. (This edge case was the source of a real correctness bug caught by our byte-exact testing, discussed in Section VI.) After filtering, `MPI_Gatherv` reassembles the full output on rank 0. `MPI_Barrier` brackets each timed region and `MPI_Wtime()` measures it, matching course-lab practice. Grayscale conversion needs no halo (each pixel is independent), so it uses a simpler scatter/gather over the RGB buffer with no exchange step.

Listing 1. Halo exchange (interior ranks)

```
MPI_Sendrecv(first_core_rows, halo_bytes,
             MPI_UNSIGNED_CHAR,
             rank - 1, TAG_UP,
             top_halo, halo_bytes, MPI_UNSIGNED_CHAR,
             rank - 1, TAG_DOWN, MPI_COMM_WORLD, &
             status);
MPI_Sendrecv(last_core_rows, halo_bytes,
             MPI_UNSIGNED_CHAR,
             rank + 1, TAG_DOWN,
             bottom_halo, halo_bytes, MPI_UNSIGNED_CHAR,
             rank + 1, TAG_UP, MPI_COMM_WORLD, &
             status);
```

The MPI primitives used throughout are `MPI_Init/Finalize`, `MPI_Comm_rank/size`, `MPI_Bcast` (image dimensions), `MPI_Scatterv`, `MPI_Sendrecv`, `MPI_Gatherv`, `MPI_Barrier`, and `MPI_Wtime` [13], [14].

IV. Experimental Setup

Hardware. All experiments ran on a single physical host: an Intel® Core™ i5-8365U CPU @ 1.60 GHz (4 physical cores, 2 threads/core via Hyper-Threading, 8 logical cores total), 32 KiB L1d and 32 KiB L1i cache per core, 256 KiB L2 cache per core, and 6 MiB shared L3 cache, with 7.6 GiB of RAM available to the test environment – a cache/memory hierarchy whose effect on multithreaded performance is well documented in standard computer-architecture references [11], [12]. The machine has a single NUMA node, so all MPI ranks communicate over shared memory rather than a real network fabric; per the course FAQ allowance for limited physical nodes, every MPI configuration ($np = 1, 2, 4, 8$) was run with `mpirun` – oversubscribe on this one host.

Software. Ubuntu 24.04.4 LTS (via WSL2 on Windows 11), GCC 13.3.0 (-O3 -Wall -Wextra), glibc/NPTL Pthreads, GCC’s libgomp OpenMP runtime (OpenMP 4.5/5.0 API, invoked with -fopenmp), and Open MPI 4.1.6 (mpicc/mpirun) [15], [33]. Plotting only (not the timed experiments themselves) uses Python 3.12 with NumPy [31] and Matplotlib [30].

Dataset. Rather than benchmark on synthetic random pixels – which we argue is methodologically weak for convolution filters, since blurring or edge-detecting uncorrelated noise produces visually and numerically meaningless output – all experiments use two real, license-clean photographs from the scikit-image sample-data distribution [28]: a 512×512 grayscale photograph (a classic image-processing benchmark image, distributed as camera) and a 512×512 RGB photograph (astronaut). A custom C program, `data/input_generator.c`, bilinearly resizes these two source images to nine resolutions for the strong-scaling / problem-size sweep – 64×64 (4,096 px) up to 5760×3240 (18,662,400 px), spanning the required 10^3 – 10^7 -pixel range – and to four further resolutions for the weak-scaling sweep, holding rows-per-worker fixed at 270 as worker count $P \in \{1, 2, 4, 8\}$ grows (1920×270 up to 1920×2160).

Second real dataset. To test the four implementations at multi-thousand-image scale rather than at multi-million-pixel scale, we additionally use Imagenette [29],

a 10-class, real-photograph subset of ImageNet distributed by fastai under the Apache-2.0 license. `data/prepare_real_dataset.py` takes 500 real photographs from each of the 10 classes (tench, English springer, cassette player, chainsaw, church, French horn, garbage truck, gas pump, golf ball, parachute) – 5,000 images total, comfortably exceeding the 5,000-image scale we targeted – resizes each to a fixed 160×160 so every image in the batch costs the same amount of filter work, and writes them in the same NetPBM format the C pipeline already reads. This is a one-time, Python-only preparation step (it does not run on the timed path); the four C programs themselves gained a `--dataset <dir> <count>` batch mode that loops over the directory inside one process (or one `mpirun` session), reusing each implementation’s existing per-image filter and parallelization code unchanged, and reports the summed time across the whole batch.

Worker counts. Every parallel configuration is run at $P \in \{1, 2, 4, 8\}$ workers (threads for Pthreads/OpenMP, ranks for MPI), the minimum sweep required by the course guide.

V. Results

Correctness. Every Pthreads, OpenMP, and MPI configuration at every worker count was compared byte-for-byte (`cmp -s`) against the sequential baseline’s output for all four filters, on both the single-image dataset (48 configurations) and a 50-image subset of the real-dataset batch mode (48 more): all 96 configurations are identical with zero byte difference (Section VI discusses one real bug this caught during development).

Strong scaling. Figure 1 shows speedup $S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$ for the 5×5 Gaussian blur on the fixed 3840×2160 image. All three models peak at $p = 4$ and then regress at $p = 8$: OpenMP rises from $0.82\times$ ($p = 1$) to $2.69\times$ ($p = 4$) before falling back to $1.67\times$ ($p = 8$); MPI rises to $1.49\times$ at $p = 4$ before falling to $1.07\times$ at $p = 8$; Pthreads is the weakest model throughout, only crossing parity at the highest worker count tested ($0.31\times$ at $p = 1$ rising monotonically to $1.15\times$ at $p = 8$). Across the full $4 \times 4 \times 3$ grid of filters, models, and worker counts (Table I), the single best result in this controlled experiment is not Gaussian blur: OpenMP reaches $3.66\times$ for Sobel edge detection at $p = 4$, then falls to $1.94\times$ at $p = 8$ – the same peak-at-4/regress-at-8 pattern, just more pronounced.

Weak scaling. Figure 2 holds work-per-worker fixed (270 rows) and grows the image with P . A perfectly weak-scaling implementation would show a flat line. OpenMP stays both the flattest and the lowest curve through $P = 4$ (0.006s at $P = 1$ to 0.008s at $P = 4$) before rising to 0.014s at $P = 8$. MPI starts close to OpenMP’s curve (0.005s at $P = 1$) but its cost grows fastest of the three, rising more than $6\times$ to 0.037s at $P = 8$ and overtaking Pthreads. Pthreads has the highest fixed cost at low P (visible even at $P = 1$, 0.021s , roughly $4\times$ OpenMP’s $P = 1$ cost) but grows the least in relative terms, ending

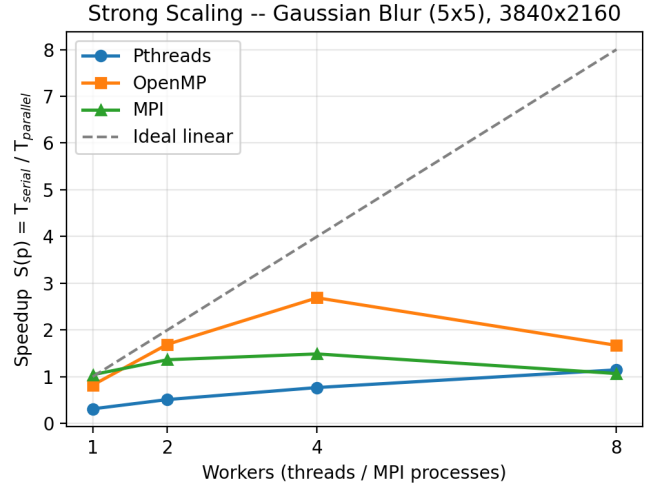


Fig. 1. Strong scaling: speedup vs. workers, Gaussian blur, 3840×2160 .

at 0.047s at $P = 8$ – consistent with the same $p=4 \rightarrow 8$ regression seen in strong scaling hitting OpenMP and MPI harder than Pthreads, which was already starting from a worse baseline.

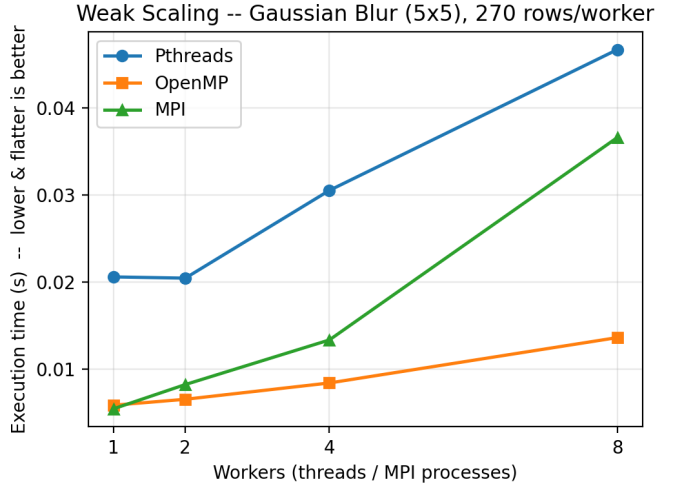


Fig. 2. Weak scaling: execution time vs. workers, 270 rows/worker fixed.

Problem-size sweep. Figure 3 fixes all three parallel models at 8 workers and varies image size from 4,096 to 18,662,400 pixels, against the sequential baseline. All three models are well behind sequential at the smallest sizes – their fixed per-call overhead dominates a few thousand pixels of work regardless of worker count – and the ranking is noisy in the middle of the range (each model briefly leads at different sizes, e.g. MPI at 1024×1024 , OpenMP at 2560×1440), reflecting real measurement variance on a virtualized, shared-cache host rather than a clean monotonic trend. At our largest tested size (18.66M

pixels), OpenMP is clearly ahead ($3.47\times$), Pthreads is essentially tied with sequential ($1.01\times$), and MPI sits in between ($1.84\times$) – i.e. only OpenMP convincingly and consistently outruns the sequential baseline once images are large enough, even at 8 workers on this 4-physical-core machine.

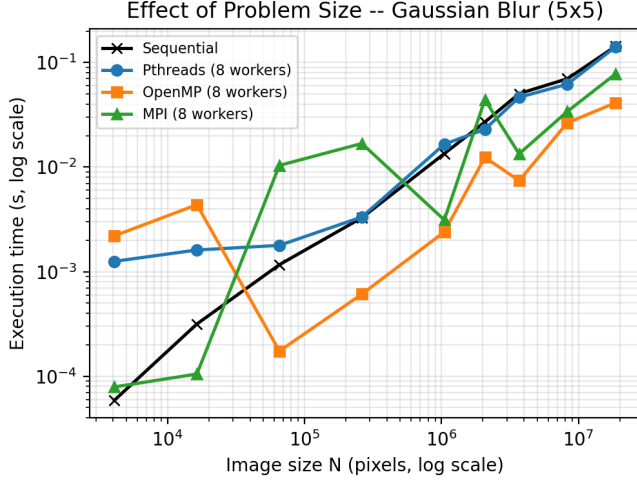


Fig. 3. Execution time vs. problem size N (log-log), 8 workers, Gaussian blur.

Execution time table. Table I reports raw execution time (seconds) for all four filters, all three models, and all four worker counts on the fixed 3840×2160 image, plus the sequential baseline.

TABLE I
Execution time (s), 3840×2160 , all filters and configurations.

Model	Workers	Gauss.	Sobel	Sharpen	Gray.
Sequential	1	0.0703	0.0469	0.0116	0.0258
Pthreads	1	0.2249	0.0656	0.0412	0.0366
Pthreads	2	0.1375	0.0412	0.0246	0.0205
Pthreads	4	0.0915	0.0256	0.0166	0.0194
Pthreads	8	0.0613	0.0224	0.0200	0.0149
OpenMP	1	0.0858	0.0540	0.0138	0.0313
OpenMP	2	0.0415	0.0292	0.0079	0.0163
OpenMP	4	0.0262	0.0128	0.0056	0.0079
OpenMP	8	0.0421	0.0242	0.0037	0.0247
MPI	1	0.0674	0.0501	0.0252	0.0306
MPI	2	0.0516	0.0300	0.0107	0.0234
MPI	4	0.0473	0.0405	0.0122	0.0222
MPI	8	0.0658	0.0425	0.0221	0.0381

Real-world dataset throughput. The experiments above all repeat work on a single image; Figure 4 instead measures throughput (images/second) processing all 5,000 real Imagenette photographs (Section IV) in one run per configuration, with every implementation’s existing per-image filter code looped internally rather than relaunched per image. The sequential baseline processes 3,854 images/s for Gaussian blur (each image is only 160×160 px, so this

is dominated by per-image overhead, not raw compute). At $p = 4$, OpenMP is the only model to beat sequential on this filter ($1.21\times$); at $p = 8$, every model is slower than at $p = 4$, and all three are slower than sequential: OpenMP falls to $0.31\times$, Pthreads to $0.25\times$, and MPI collapses to $0.04\times$ – $25\times$ slower than running the whole batch sequentially. The effect is most extreme for the cheapest filter, sharpening: MPI at $p = 8$ processes only 246 images/s versus sequential’s 21,874, an $89\times$ slowdown, because the per-image MPI_Bcast/Scatterv/Gatherv/Barrier round trip costs far more than the ~ 9 multiply-adds per pixel the filter itself needs at this image size. Table II reports throughput for all four filters; the pattern – a peak around $p = 2-4$ followed by a severe drop at $p = 8$ – holds for every filter and every parallel model.

Real-Dataset Throughput -- Gaussian Blur (5x5), 5000 real photos

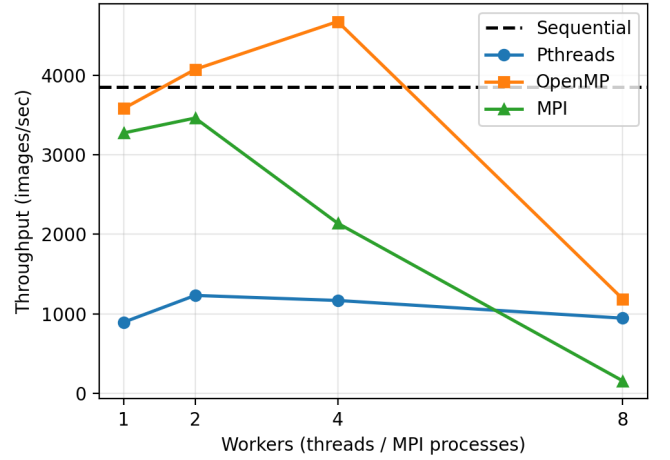


Fig. 4. Real-dataset throughput: images/sec vs. workers, Gaussian blur, 5,000 real Imagenette photographs.

TABLE II
Throughput (images/s), 5,000 real photographs, 160×160 each.

Model	Workers	Gauss.	Sobel	Sharpen	Gray.
Sequential	1	3853.7	5444.9	21873.8	9740.2
Pthreads	1	894.7	2107.4	2643.9	3348.6
Pthreads	2	1230.9	2268.8	2600.7	2953.1
Pthreads	4	1167.4	1715.5	1845.1	1926.8
Pthreads	8	944.6	1094.1	1122.5	1147.8
OpenMP	1	3584.7	5078.3	20222.0	10395.1
OpenMP	2	4075.4	7290.0	28510.2	14695.4
OpenMP	4	4677.2	13433.3	42820.7	26799.6
OpenMP	8	1186.9	1926.9	3100.4	3335.8
MPI	1	3276.0	4642.8	17789.9	9376.3
MPI	2	3463.0	4724.0	15634.6	8714.5
MPI	4	2139.7	4164.7	8248.2	6209.8
MPI	8	155.9	217.5	245.8	327.1

VI. Discussion

Why Pthreads is consistently the weakest model. At $p = 1$, Pthreads is slower than sequential for all four filters

– Gaussian blur $3.2\times$ slower (0.225 s vs. 0.070 s), Sobel $1.4\times$, sharpening $3.5\times$, grayscale $1.4\times$ – despite doing identical work on one worker thread. The only difference is one `pthread_create/pthread_join` pair, but our design creates and destroys a fresh pool of OS threads for every filter call rather than reusing a persistent pool; OpenMP at $p = 1$ stays far closer to sequential across the board (e.g. Gaussian blur is only 18% slower) because libgomp’s thread pool, once created, is reused across all four filter calls in the same process. As worker count grows, Pthreads does eventually cross parity for three of the four filters – Gaussian blur at $p = 8$ ($1.15\times$), Sobel from $p = 2$ onward (up to $2.09\times$ at $p = 8$), grayscale from $p = 2$ onward (up to $1.72\times$ at $p = 8$) – but sharpening, the cheapest filter, never recovers: it peaks at only $0.70\times$ at $p = 4$ and falls back to $0.58\times$ at $p = 8$, because its ~ 9 -multiply-add-per-pixel cost is too small for any worker count to amortize eight repeated thread-creation calls. This matches the general observation that thread-pool reuse, not merely parallel decomposition, is often the dominant factor in shared-memory performance [17]. A separate, well-known shared-memory bottleneck, false sharing of cache lines between threads writing to nearby addresses [23], [24], is not the cause here: every thread writes to a wholly disjoint, contiguous slice of the output buffer, so the dominant cost we measure is thread lifecycle overhead, not cache-line contention.

Why MPI’s fixed per-rank cost shows up even at $p = 1$. MPI sharpening at $p = 1$ (0.025 s) is $2.2\times$ slower than sequential despite doing no inter-rank communication at all, purely from MPI_Init, communicator setup, and a Scatterv/Gatherv pair that still copies data locally with one rank. Sharpening is also the only filter for which MPI never decisively beats sequential at any worker count (peaking at $1.09\times$ at $p = 2$, falling to $0.53\times$ by $p = 8$) – the same overhead-to-work imbalance seen in Pthreads, since both models pay a fixed per-call cost that the lightest filter has the least work available to absorb.

Why OpenMP wins overall, but still regresses at $p = 8$ for most filters. OpenMP reaches the best result of any single-image experiment ($3.66\times$, Sobel, $p = 4$) and is the only model that clearly separates from the sequential baseline at our largest tested image size in Figure 3 ($3.47\times$ at 18.66M pixels). Even so, three of its four filters peak at $p = 4$ and fall back at $p = 8$ (Gaussian blur $2.69 \rightarrow 1.67\times$; Sobel $3.66 \rightarrow 1.94\times$; grayscale $3.25 \rightarrow 1.04\times$) – only sharpening keeps improving all the way to $p = 8$ ($3.14\times$, its best result). Our test CPU has only 4 physical cores; the 8 logical cores come from Hyper-Threading, which shares execution units and L1/L2 cache between the two threads on a core, so going from 4 to 8 OpenMP threads adds contention for shared resources rather than genuinely new execution capacity for most of these filters – consistent with the Roofline view that performance saturates once a kernel’s working set exceeds what a shared cache/memory path can deliver [25], and with reported NUMA/shared-

cache degradation under core oversubscription [26]. We use static scheduling throughout (Section III); dynamic loop scheduling [27] could rebalance uneven per-thread load, but would not address contention for shared execution units between Hyper-Threaded sibling threads, which is the mechanism behind the regression observed here. The same $p=4 \rightarrow 8$ regression appears, less uniformly, in Pthreads and MPI too (Table I), but the signal is noisier there because their absolute speedups are smaller to begin with on this virtualized, shared-cache host.

Why the real-dataset throughput collapse at $p = 8$ is so much more dramatic. The single-image experiments above only hint at a $p=4 \rightarrow 8$ regression, partly obscured by run-to-run measurement noise. Repeating the same per-call overhead 5,000 times instead of once (Table II) removes that ambiguity: every one of our 12 model/filter combinations is slower at $p = 8$ than at $p = 4$, several catastrophically. MPI is the most extreme: Gaussian-blur throughput falls from 2,140 images/s at $p = 4$ to just 156 images/s at $p = 8$ ($13.7\times$ slower), and sharpening falls from 8,248 to 246 images/s ($33.6\times$ slower) – $89\times$ slower than the sequential baseline outright. Each of the 5,000 images independently triggers a full MPI_Bcast/Scatterv/halo-exchange/ Gatherv/Barrier round trip across 8 processes on a 4-physical-core host; at 160×160 px there is only $\sim 25,600$ pixels of real work per image; once 8-way oversubscription is reached, the collective-communication latency itself – paid out 5,000 times – dominates totally, consistent with reported collective-operation overhead scaling with rank count [16]. OpenMP and Pthreads degrade far less severely (OpenMP Gaussian blur falls $3.94\times$, Pthreads only $1.24\times$, from $p = 4$ to $p = 8$) because their per-call cost is a thread-pool/region launch rather than a multi-process collective operation, but the direction of the effect – and the fact that it appears for every filter in every model – confirms that the noisy single-image signal above was real, not an artifact: 8-way parallelism on 4 physical cores reliably costs more than it returns once per-call overhead, not raw compute, is the bottleneck, and that effect compounds linearly with how many times the call is repeated.

Why small images favor the sequential version. Figure 3 shows all three parallel models well behind sequential at the smallest sizes tested, only reaching parity in the 10^5 – 10^6 pixel range. This is Amdahl’s law made concrete [3]: fixed per-call overhead (thread/process creation, scatter/gather, barriers) does not shrink with problem size, so for a small enough image it exceeds the actual filtering work, regardless of how many workers are available – the same mechanism, at a single-image scale, that the dataset-throughput experiment exposes at a 5,000-image scale. Gustafson’s framing [4] – that parallelism pays off when the problem grows with the machine – is reflected in our weak-scaling curves (Fig. 2): OpenMP’s curve grows the least in absolute terms through $p = 4$, while every model’s cost still rises from $p = 4$ to $p = 8$, the same regression

seen throughout this section.

Programming effort. We measured effort directly via source line count relative to the 357-line sequential baseline (which now includes each version’s dataset-throughput batch mode, Section IV): OpenMP is 338 lines (−5%), Pthreads is 356 lines (−0.3%), and MPI is 512 lines (+43%). The batch-mode addition itself is conceptually trivial for every version – a loop over the dataset directory that calls each implementation’s existing per-image entry point unchanged – so it shifts every line count by roughly the same handful of lines without changing the relative ordering established by the original four single-image filters: OpenMP’s directive model still reuses the exact loop bodies from Algorithm 1 verbatim; Pthreads still needs only an explicit task struct and four worker functions, no new concepts beyond the sequential algorithm; MPI is still by far the largest, because its batch loop must also re-broadcast per-image dimensions and rebuild scatter/gather distribution tables on every single iteration – a direct illustration of the same per-call bookkeeping overhead responsible for its throughput collapse above. MPI’s larger size is also a direct illustration of a real correctness bug our byte-exact testing caught and that the other two models cannot exhibit by construction: our first MPI implementation computed a mirrored-halo convolution result for the true top/bottom edge rows of the image instead of leaving them untouched like the other three versions, silently disagreeing with the sequential baseline at every image border until corrected (Section III). Aggregate min/max/mean pixel statistics – the check used in this project’s earlier MPI-only phase – did not surface this bug; only direct byte comparison did, which we recommend over statistical checks for any future phase of this project.

VII. Conclusion

We implemented and verified four versions – sequential, Pthreads, OpenMP, and MPI – of a four-filter image-processing pipeline (Gaussian blur, Sobel edge detection, sharpening, grayscale conversion), benchmarked on two complementary real datasets: nine resolutions of two photographs spanning 4,096 to 18,662,400 pixels, and a balanced 5,000-photograph subset of Imagenette. OpenMP gave the best single-image result observed in this study, a $3.66\times$ speedup for Sobel edge detection at 4 threads, and was the only model that clearly and consistently beat sequential at our largest tested image size. A single, unifying pattern explains most of what we observed: this 4-physical-core machine rewards parallelism up to about 4 workers and then increasingly punishes it at 8, because the second Hyper-Threaded sibling on each core adds contention, not real capacity. That pattern is only a noisy hint in the single-image experiments, but the 5,000-image dataset throughput experiment removes the ambiguity entirely: every one of 12 model/filter combinations is slower at 8 workers than at 4, and MPI’s collapse is

severe enough to be unmistakable – $13\text{--}34\times$ slower from $p = 4$ to $p = 8$, and up to $89\times$ slower than sequential outright – because each small image independently pays the full cost of an MPI collective-communication round trip across 8 oversubscribed processes. Pthreads’ weakest filter (sharpening) never recovers from this overhead at any scale we tested. As future work, the most direct next step – and the one that would realize this project’s original hybrid design [18] – is to nest an OpenMP parallel for inside each MPI rank’s row-strip, so MPI handles only inter-rank communication while OpenMP exploits all cores within a rank; our dataset-throughput results suggest an equally important second step specific to batch workloads is to amortize MPI’s per-call setup (broadcast, scatter/gather table construction) once across a whole batch of images rather than once per image. A third, lower-effort improvement would be to replace Pthreads’ per-filter `pthread_create/pthread_join` pattern with a persistent worker pool, closing much of the gap to OpenMP that this data shows despite both using an identical decomposition. A more ambitious direction would move the kernel itself to a GPU via CUDA [32], trading this project’s CPU-core-count limits for a much larger pool of lightweight cores at the cost of an explicit host/device data-transfer step.

References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [2] M. J. Flynn, “Some computer organizations and their effectiveness,” *IEEE Trans. Comput.*, vol. C-21, no. 9, pp. 948–960, Sep. 1972.
- [3] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.
- [4] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Commun. ACM*, vol. 31, no. 5, pp. 532–533, May 1988.
- [5] D. Jaiswal and P. Kumar, “A survey on parallel computing for traditional computer vision,” *Concurrency Computat. Pract. Exper.*, vol. 34, no. 4, p. e6638, 2022.
- [6] N. S. Haron, R. Amir, I. B. A. Aziz, L. T. Jung, and S. R. M. Shukri, “Parallelization of edge detection algorithm using MPI on Beowulf cluster,” in *Innovations in Computing Sciences and Software Engineering*. Springer, 2010, pp. 477–482.
- [7] P. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011.
- [8] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Boston, MA: Addison-Wesley, 2003.
- [9] I. Foster, *Designing and Building Parallel Programs*. Boston, MA: Addison-Wesley, 1995.
- [10] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*. New York: McGraw-Hill, 2003.
- [11] D. E. Culler, J. P. Singh, and A. Gupta, *Parallel Computer Architecture: A Hardware/Software Approach*. San Francisco, CA: Morgan Kaufmann, 1999.
- [12] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA: Morgan Kaufmann, 2017.
- [13] W. Gropp, E. Lusk, and A. Skjellum, *Using MPI: Portable Parallel Programming with the Message-Passing Interface*, 2nd ed. Cambridge, MA: MIT Press, 1999.
- [14] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard, Version 4.1,” Nov. 2023.

- [15] E. Gabriel et al., “Open MPI: Goals, concept, and design of a next generation MPI implementation,” in Proc. 11th European PVM/MPI Users’ Group Meeting, Budapest, Hungary, 2004, pp. 97–104.
- [16] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, 2005.
- [17] R. Rabenseifner, G. Hager, and G. Jost, “Hybrid MPI/OpenMP parallel programming on clusters of multi-core SMP nodes,” in Proc. 17th Euromicro Int. Conf. Parallel, Distributed and Network-Based Processing, 2009, pp. 427–436.
- [18] R. Rabenseifner, G. Hager, G. Jost, and R. Keller, “Hybrid MPI and OpenMP parallel programming,” in *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, LNCS vol. 4192. Springer, 2006.
- [19] L. Dagum and R. Menon, “OpenMP: an industry-standard API for shared-memory programming,” *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, 1998.
- [20] OpenMP Architecture Review Board, “OpenMP Application Programming Interface, Version 5.2,” Nov. 2021.
- [21] B. Chapman, G. Jost, and R. van der Pas, *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: MIT Press, 2007.
- [22] IEEE, “IEEE Std 1003.1c-1995, Information Technology – POSIX – Threads Extension,” Institute of Electrical and Electronics Engineers, 1995.
- [23] W. J. Bolosky and M. L. Scott, “False sharing and its effect on shared memory performance,” in Proc. USENIX Symp. Experiences with Distributed and Multiprocessor Systems (SEDMS IV), San Diego, CA, 1993.
- [24] T. Liu and E. D. Berger, “SHERIFF: Detecting and eliminating false sharing,” Univ. of Massachusetts Amherst, Tech. Rep. UMC-CS-2010-047, 2010.
- [25] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.
- [26] Z. Majó and T. R. Gross, “(Mis)understanding the NUMA memory system performance of multithreaded workloads,” in Proc. IEEE Int. Conf. Cluster Computing, 2013, pp. 1–8.
- [27] F. M. Ciorba, “Dynamic load balancing of loops in OpenMP,” SC19 OpenMP Tech Talk, Univ. of Basel, 2019.
- [28] S. van der Walt et al., “scikit-image: image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014.
- [29] J. Howard, “Imagenette: a smaller subset of 10 easily classified classes from ImageNet,” fastai, GitHub repository, 2019. [Online]. Available: <https://github.com/fastai/imagenette>
- [30] J. D. Hunter, “Matplotlib: a 2D graphics environment,” *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.
- [31] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.
- [32] NVIDIA Corporation, “CUDA C++ Programming Guide,” Santa Clara, CA, 2024.
- [33] The Open MPI Project, “Open MPI Documentation, Version 4.1,” 2023.